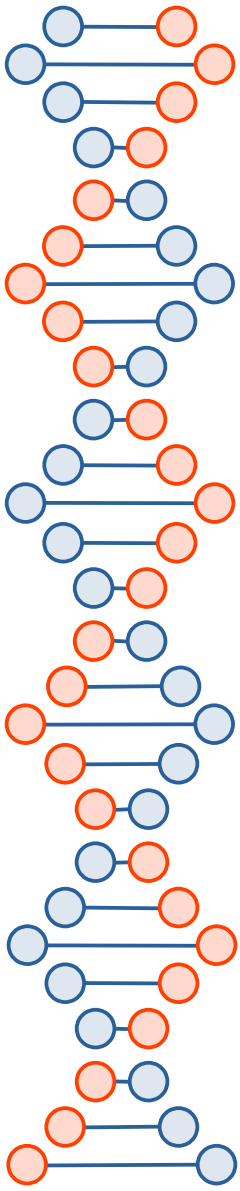
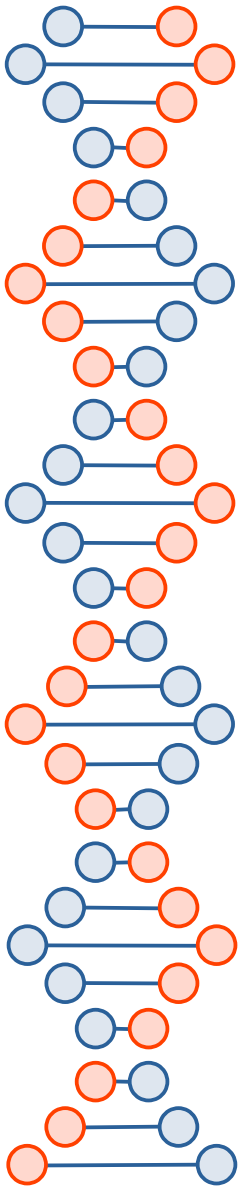


Modelos Baseline:
Algoritmo de predicción aleatoria y
Algoritmo de regla cero
Dr. Josafá Pontes
Machine Learning



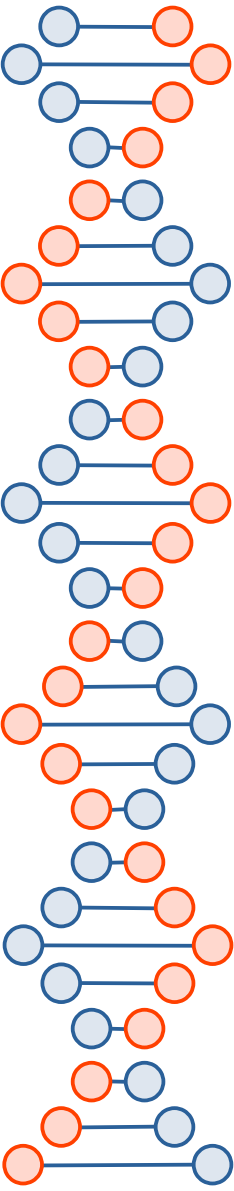
Outline

- How to implement the random prediction algorithm.
- How to implement the zero rule prediction algorithm.



Overview

- It is important to establish baseline performance on a predictive modeling problem.
- A baseline provides a point of comparison for the more advanced methods that you evaluate later.
- In this tutorial, you will discover how to implement baseline machine learning algorithms from scratch.

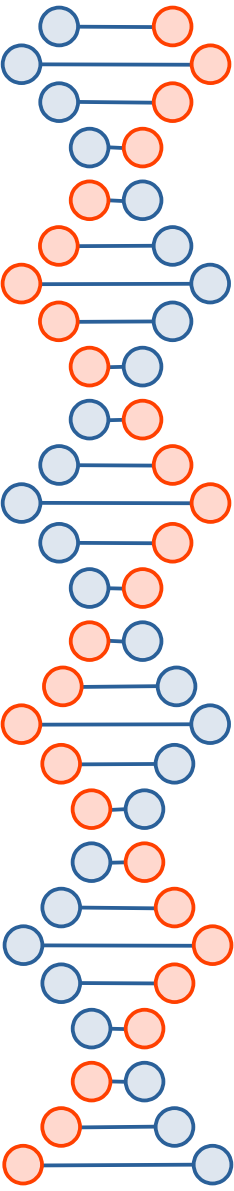


Baseline methods (1)

- There are many machine learning algorithms to choose from.
- How do you know whether the predictions for a given algorithm are good or not?
- The answer is to use a baseline prediction algorithm.
- A baseline prediction algorithm provides a set of predictions that you can evaluate as you would any predictions for your problem, such as classification accuracy or RMSE.
- The scores from these algorithms provide the required point of comparison when evaluating all other machine learning algorithms on your problem.
- Once established, you can comment on how much better a given algorithm is as compared to the naive baseline algorithm, providing context on just how good a given method actually is.

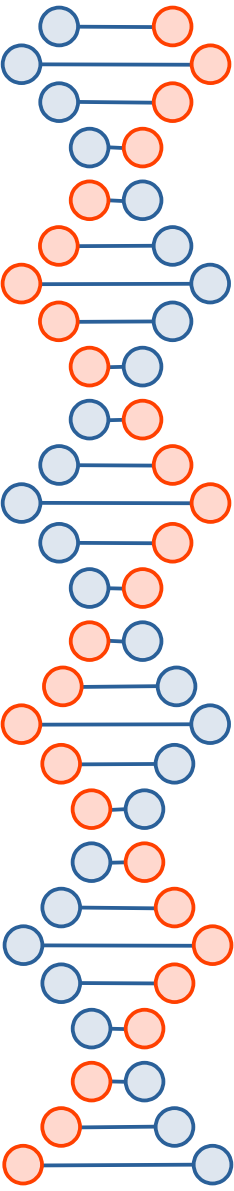
Baseline methods (2)

- The two most commonly used baseline algorithms are:
 - Random Prediction Algorithm.
 - Zero Rule Algorithm.
- When starting on a new problem that is more difficult than a conventional classification or regression problem, it is a good idea to first devise a random prediction algorithm that is specific to your prediction problem.
- Later you can improve upon this and devise a zero rule algorithm. Let's implement these algorithms and see how they work.



Random Prediction Algorithm (1)

- The random prediction algorithm predicts a random outcome as observed in the training data.
- It is perhaps the simplest algorithm to implement. It requires that you store all of the distinct outcome values in the training data, which could be large on regression problems with lots of distinct values.
- Because random numbers are used to make decisions, it is a good idea to fix the random number seed prior to using the algorithm.
- This is to ensure that we get the same set of random numbers, and in turn the same decisions each time the algorithm is run.



Random Prediction Algorithm (2)

- Below is an implementation of the Random Prediction Algorithm in a function named `random algorithm()`.
- The function takes both a training dataset that includes output values and a test dataset for which output values must be predicted.
- The function will work for both classification and regression problems.
- It assumes that the output value in the training data is the final column for each row.

Random Prediction Algorithm (3)

- First, the set of unique output values is collected from the training data.
- Then a randomly selected output value from the set is selected for each row in the test set.
- We can test this function with a small dataset that only contains the output column for simplicity.
- The output values in the training dataset are either 0 or 1, meaning that the set of predictions the algorithm will choose from is (0, 1).
- The test set also contains a single column, with no data as the predictions are not known.
- The random prediction algorithm is easy to implement and fast to run, but we could do better as a baseline.

Random Prediction Algorithm (4)

Example of Making Random Predictions

```
use strict;  
use warnings;  
use Data::Dump qw(dump);  
use List::Util qw(zip min max sum);
```

Example of Making Random Predictions

Defined in Section 5.2.1 Random Prediction Algorithm

Example of Making Random Predictions

Generate random predictions

```
sub random_algorithm{  
    my ($train, $test) = @_;  
    my $output_values = [map {$_->[-1]} @$train];  
    tie my %unique, 'Tie::IxHash';  
    my @unique = grep{defined $_} %unique = map {$_ => undef} @$output_values;  
    my @predicted = ();  
    for (@$test){  
        my $index = int(rand(@unique));  
        push @predicted, $unique[$index];  
    }  
    return \@predicted;  
}
```

```
rand(1);
```

```
my $train = [[0], [1], [0], [1], [0], [1]];
```

```
my $test = [[undef], [undef], [undef], [undef]];
```

```
my $predictions = random_algorithm($train, $test);
```

```
print dump $predictions;
```

Example Output From Making Random Predictions.

```
# [0, 0, 1, 0]
```

Zero Rule Algorithm

- The Zero Rule Algorithm is a better baseline than the random algorithm.
- It uses more information about a given problem to create one rule in order to make predictions.
- This rule is different depending on the problem type.
- Let's start with classification problems, predicting a class label.

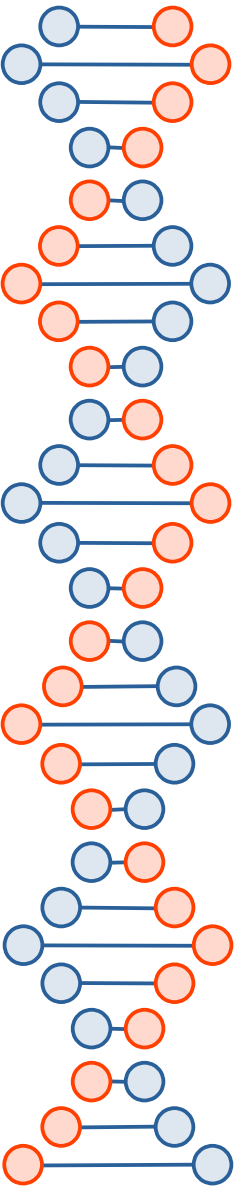
Zero Rule Algorithm – Classification (1)

- For classification problems, the one rule is to predict the class value that is most common in the training dataset.
- This means that if a training dataset has 90 instances of class 0 and 10 instances of class 1 that it will predict 0 and achieve a baseline accuracy of 90/100 or 90%.
- This is much better than the random prediction algorithm that would only achieve 82% accuracy on average.
- For details on how this is estimate for random search is calculated, see below:

$$= ((0.9 \times 0.9) + (0.1 \times 0.1)) \times 100 = 82\%$$

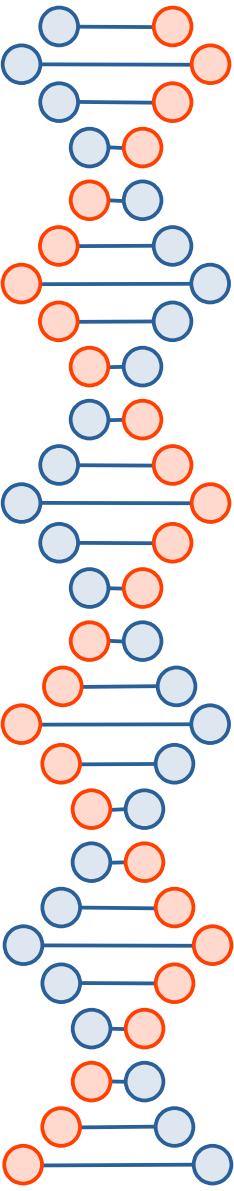
- Below is a function named `zero rule algorithm classification()` that implements this for the classification case.

Zero Rule Algorithm – Classification (2)



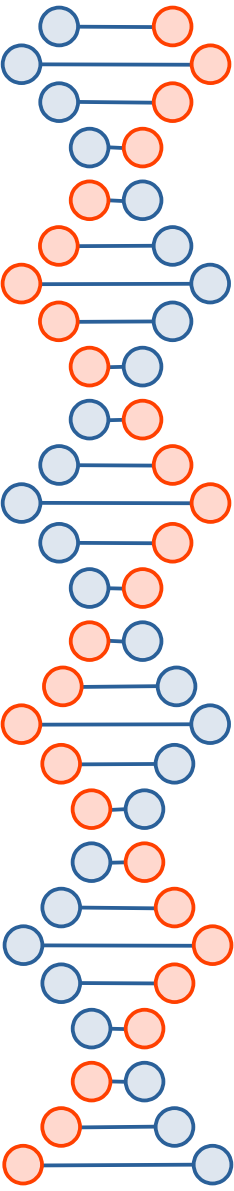
```
# Function To Make Zero Rule Classification Predictions.
# zero rule algorithm for classification
sub zero_rule_algorithm_classification{
  my ($self, $train, $test) = @_;
  my @output_values = map {$_->[-1]} @$train;
  my %counter      = ();
  map {$counter{"$_"}++} @output_values; # Counts
  my $prediction   = $counter{max map {$_} %counter = reverse %counter};
  return [($prediction) x scalar(@$test)]; # predictions
}

sml->add_to_class('zero_rule_algorithm_classification', \&'zero_rule_algorithm_classification');
srand(1);
my $train = [['0'], ['0'], ['0'], ['0'], ['1'], ['1']];
my $test  = [[undef], [undef], [undef], [undef]];
my $predictions = sml->zero_rule_algorithm_classification($train, $test);
print dump $predictions;
# Example of Zero Rule Classification Predictions.
# [0, 0, 0, 0]
```



Zero Rule Algorithm – Classification (3)

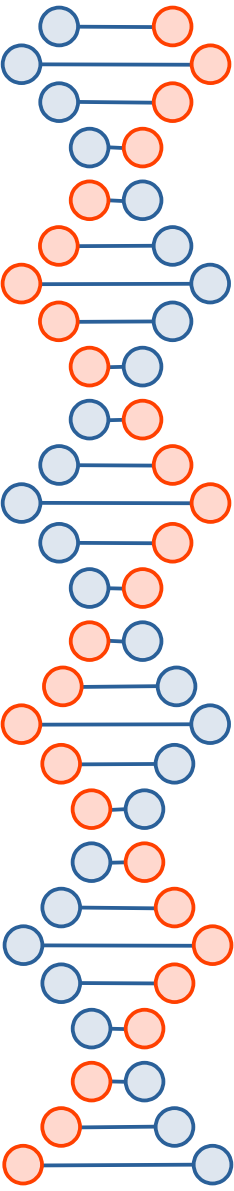
- The function makes use of the `max()` function with the key attribute, which is a little clever.
- Given a list of class values observed in the training data, the `max()` function takes a set of unique class values and calls the `count` on the list of class values for each class value in the set.
- The result is that it returns the class value that has the highest count of observed values in the list of class values observed in the training dataset.
- If all class values have the same count, then we will choose the first class value observed in the dataset.
- Once we select a class value, it is used to make a prediction for each row in the test dataset.
- Below is a worked example with a contrived dataset that contains 4 examples of class 0 and 2 examples of class 1.
- We would expect the algorithm to choose the class value 0 as the prediction for each row in the test dataset.



Zero Rule Algorithm – Regression (1)

- Regression problems require the prediction of a real value.
- A good default prediction for real values is to predict the central tendency. This could be the mean or the median.
- A good default is to use the mean (also called the average) of the output value observed in the training data.
- This is likely to have a lower error than random prediction which will return any observed output value.
- Below is a function to do that named zero rule algorithm regression().
- It works by calculating the mean value for the observed output values.

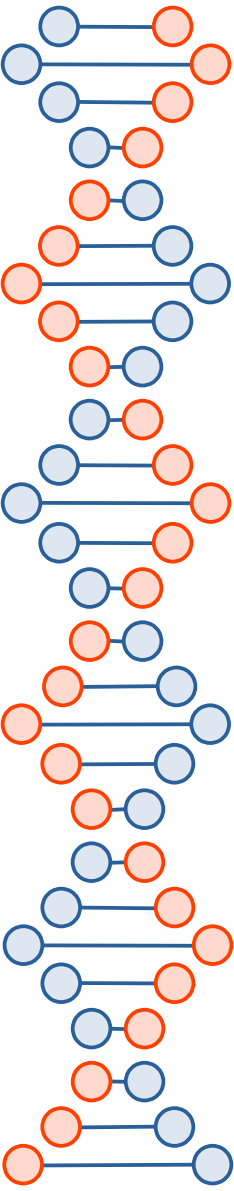
$$mean = \frac{\sum_{i=1}^n value_i}{count(values)}$$



Zero Rule Algorithm – Regression (2)

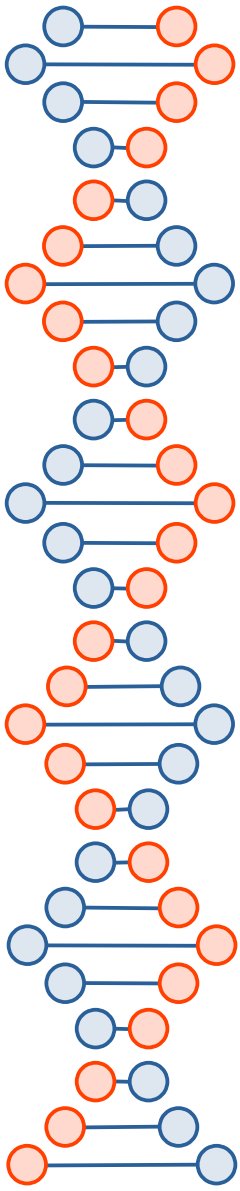
- Once calculated, the mean is then predicted for each row in the training data.
- This function can be tested with a simple example.
- We can contrive a small dataset where the mean value is known to be 15.
- Below is the complete example.
- We would expect that the mean value of 15 will be predicted for each of the 4 rows in the test dataset.
- Running the example calculates the predicted output values that are printed.
- As expected, the mean value of 15 is predicted for each row in the test dataset.

Zero Rule Algorithm – Regression (3)



```
# Example of Zero Rule Regression Predictions
# Function To Make Zero Rule Regression Predictions.
# zero rule algorithm for regression
my $zero_rule_algorithm_regression = sub{
  my ($self, $train, $test) = @_;
  my $output_values = [map {$_->[-1]} @$train];
  my $prediction = sprintf '%0.1f', sum(@$output_values) / @$output_values;
  my $predicted = [map {$prediction} (0 .. $$test)];
  return $predicted;
};

sml->add_to_class('sml', 'zero_rule_algorithm_regression', $zero_rule_algorithm_regression);
srand(1);
$train = [[10], [15], [12], [15], [18], [20]];
$test = [[undef], [undef], [undef], [undef]];
my $predictions = sml->zero_rule_algorithm_regression($train, $test);
print dump $predictions;
# Example Output From Making Zero Rule Regression Predictions.
# [15.0, 15.0, 15.0, 15.0]
```

Summary

- In this tutorial, you discovered the importance of calculating a baseline of performance on your machine learning problem.
- You now know:
 - How to implement a random prediction algorithm for classification and regression problems.
 - How to implement a zero rule algorithm for classification and regression problems.